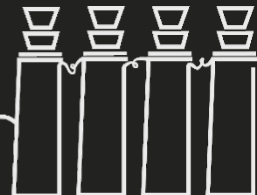


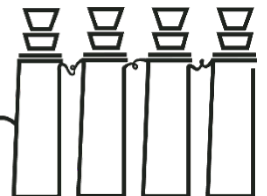
TRABALHO PRÁTICO 01 LISTA ENCADEADA

<Aluno 1> – <matrícula>
<Aluno 2> – <matrícula>



O problema

- Uma explosão no laboratório da AEDS fez os Pokémon fugirem do Centro de Pesquisa.
- Dois treinadores saem para resgatá-los. A cada Pokémon, vai o que estiver mais perto dele.
- Cada captura gasta uma Pokébola. Sem nenhuma, o treinador volta ao Centro, entrega o que pegou e recarrega.
- No fim, o Centro grava um relatório em .txt com os Pokémon recuperados.
- Restrição do enunciado: toda lista tem que ser encadeada, e cada entidade tem que ser um TAD.

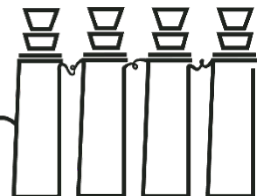


Modelagem do problema

- Começamos pelos TADs, de baixo para cima. Cada um conhece só o de baixo:

Pokémon → PokeLista → Treinador → Centro de Pesquisa

- O Pokémon tem Id, número da Pokédex, nome, tipo e localização. A PokeLista é a lista encadeada desses Pokémon.
- O Treinador tem uma PokeLista dentro dele, com o que já capturou. O Centro tem duas: os fugitivos e os recuperados.
- A missão fica num módulo separado, e o main só prepara o sorteio e chama o menu. A dependência nunca volta, então não há inclusão circular.
- <Aluno 1> ficou com <...> e <Aluno 2> com <...>.



A lista encadeada, com célula cabeça

```
include/conexao.h

13  typedef struct conec {
14      Pokemon pokemon;
15      struct conec *prox;
16  } conec;
```

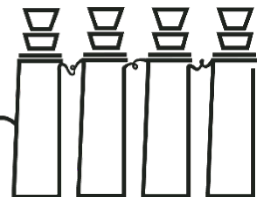
Figura 01 – A célula, em include/conexao.h

```
include/pokelista.h

13  typedef struct {
14      conec *primeiro;
15      conec *ultimo;
16      int tamanho;
17  } Pokelista;
```

Figura 02 – A lista, em include/pokelista.h

- A célula guarda o Pokémon por valor, não um apontador. Passar um Pokémon de uma lista para outra é copiar para fora e copiar para dentro: nenhuma memória fica compartilhada entre as listas.
- O nome struct conec é obrigatório, porque o campo prox aponta para o próprio tipo — e dentro do typedef o nome Pokelista ainda não existe.
- A célula cabeça não guarda Pokémon nenhum. Ela existe para que o primeiro elemento também tenha um antecessor, e assim a remoção não precisa de caso especial. O campo ultimo faz a inserção no fim custar $O(1)$.



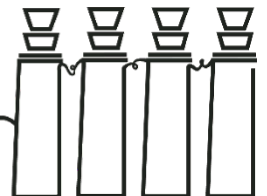
A remoção, e o apontador ultimo

src/pokelista.c

```
65     alvo = anterior->prox;
66
67     if (removido != NULL) {
68         *removido = alvo->pokemon;
69     }
70
71     /* Desliga o alvo da corrente. */
72     anterior->prox = alvo->prox;
73
74     /* Sem esta correcao, ultimo ficaria apontando para memoria liberada e a
75        proxima insercao escreveria nela. */
76     if (alvo == pl->ultimo) {
77         pl->ultimo = anterior;
78     }
79
80     free(alvo);
81     pl->tamanho--;
82
83     return 1;
84 }
```

Figura 03 – Remoção por Id, em src/pokelista.c

- O laço anda com um apontador para a célula anterior ao alvo. Numa lista encadeada simples não se volta: para desligar uma célula é preciso ter em mãos quem aponta para ela.
- O Pokémon é copiado para fora antes do free. Depois dele a memória não é mais nossa.
- Se o removido era o último, ultimo passa a apontar para o anterior (linhas destacadas).
- Sem essas duas linhas, ultimo apontaria para memória já liberada e a próxima inserção escreveria nela. Às vezes funciona, às vezes o programa quebra — é o bug clássico da lista com apontador para o fim.



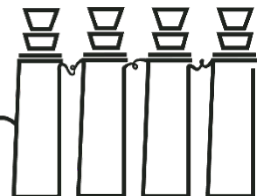
A escolha do treinador

```
src/missao.c

53 long long distanciaQuadrado(cord a, cord b)
54 {
55     long long dx = (long long) a.cordX - b.cordX;
56     long long dy = (long long) a.cordY - b.cordY;
57
58     return dx * dx + dy * dy;
59 }
```

Figura 04 – O cálculo da distância, em src/missao.c

- A missão vai para o treinador mais próximo e, em caso de empate, para o de menor identificador.
- Comparamos o quadrado da distância, sem tirar a raiz. O quadrado é um inteiro exato, então o empate é reconhecido com segurança — em ponto flutuante, dois valores que deveriam ser iguais podem diferir na última casa.
- A raiz é crescente, então comparar os quadrados dá a mesma ordem. Ela aparece só na hora de imprimir, com %.2f.
- A conta é feita em long long, e as coordenadas são limitadas a um milhão. Sem esse limite a soma estoura o long long, a raiz devolve nan e a missão vai para o treinador errado.

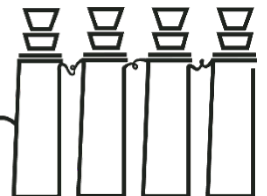


A ordem dos dois testes

```
src/missao.c
356     for (id = 1; id <= qtdFugitivos; id++) {
357         if (!pokecenterBuscarFugitivo(cp, id, &alvo)) {
358             continue;
359         }
360
361         escolhido = resgataPokemon(cp, t1, t2, &alvo);
362         if (escolhido == NULL) {
363             continue;
364         }
365
366         /* A ORDEM DESTES DOIS TESTES IMPORTA: se o Pokemon capturado era o
367            ultimo, a missao termina sem recarga, mesmo que o treinador tenha
368            ficado sem Pokebolas. E o que o exemplo da especificacao mostra. */
369         if (!pokecenterTemFugitivos(cp)) {
370             break;
371         }
372
373         if (treinadorGetPokebolas(escolhido) == 0) {
374             retornaAoCentro(cp, escolhido);
375         }
376     }
```

Figura 05 – O laço da missão, em src/missao.c

- Depois de capturar, o programa pergunta primeiro se acabaram os fugitivos; só depois se o treinador ficou sem Pokébolas.
- No exemplo do enunciado o último resgate é o do Umbreon, e ele zera as Pokébolas da Rosa. O exemplo mostra “Pokébolas restantes: 0” e em seguida o bloco de fim, sem nenhuma recarga.
- Com os dois testes trocados, a Rosa voltaria ao Centro para receber Pokébolas que não vai usar, e sairia um bloco a mais na tela.
- O laço percorre os lds de 1 até n. A busca no Centro, no começo do corpo, é o que confirma que aquele Pokémon ainda está fugido.



Execução: a entrada e a saída

testes/oficiais/teste1.txt

```
Rosa 2
Nate 2
5
610 Axew Dragão 7 7
657 Frogadier Água 10 7
387 Turtwig Folha 1 1
715 Noivern Voador -10 -10
197 Umbreon Noturno 5 7
```

Figura 06 – O arquivo de entrada

saída no terminal

```
-----
Pokémon alvo: Axew
Localização: (7,7)

Distância Treinador(a) Rosa: 9.90
Distância Treinador(a) Nate: 9.90

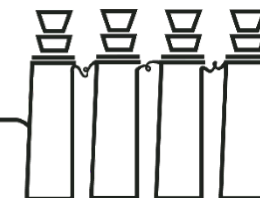
Missão atribuída ao Treinador(a) Rosa.

Treinador(a) Rosa se movimentou para (7,7).
Axew capturado com sucesso!

Pokébolas restantes para o Treinador(a) Rosa: 1
```

Figura 07 – Um resgate, no terminal

As duas primeiras linhas trazem os treinadores, com nome e quantidade de Pokébolas. Depois vem a quantidade de Pokémon e uma linha por Pokémon: número da Pokédex, nome, tipo e coordenada.



Execução: a recarga e o relatório

```
saída no terminal

=====
Treinador(a) Rosa SEM POKÉBOLAS
=====

Treinador(a) Rosa retorna ao Centro de Pesquisa.

Entregando Pokémon ao Centro de Pesquisa.

Treinador(a) Rosa recebeu 9 Pokébolas.
```

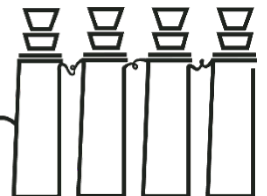
Figura 08 – O treinador fica sem Pokébolas

```
relatorio.txt

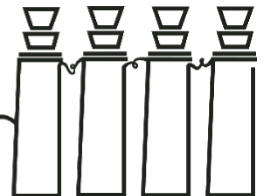
Pokemon recuperados:
610 Axew
657 Frogadier
387 Turtwig
197 Umbreon
715 Noivern
```

Figura 09 – relatorio.txt, gravado no fim

A quantidade da recarga é sorteada entre 1 e 20. O relatório sai na ordem em que os Pokémon foram entregues, porque a lista insere no fim e a entrega retira do começo.

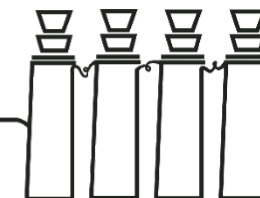


- 32 arquivos de entrada e um script que roda todos: 72 verificações, todas passando.
- A saída do exemplo do enunciado é comparada com diff, caractere por caractere.
- A recarga é sorteada, então o script fixa o sorteio numa cópia do projeto. O programa entregue não tem nada de teste dentro dele.
- 16 casos inválidos: arquivo que não existe, campo faltando, Pokébolas negativas, coordenada fora do mapa. Todos recusados com mensagem, sem travar.
- Memória: com 20 Pokémon são 64 alocações e 64 liberações; com 500, 1504 e 1504. A conta é 4 células cabeça mais 3 por Pokémon.



O que não fechava no enunciado

- O texto pede “o ID e o Nome” no relatório, mas o exemplo imprime 610 Axew — e 610 é o número da Pokédex, não o Id. Seguimos o exemplo.
- O bloco final da saída tem uma linha em branco a mais e um caractere invisível no lugar do recuo, diferente dos outros três blocos. Usamos o padrão dos outros.
- As linhas do relatório aparecem com um espaço de recuo, que é recuo de parágrafo do documento. Gravamos sem ele.
- O arquivo de entrada está em três colunas: copiar e colar junta a coordenada 7 7 em 77. Conferimos a leitura contra o arquivo de teste oficial.
- Conferimos as dez distâncias do exemplo, uma a uma. Todas batem com o enunciado.



ZIVIANI, N. Projeto de algoritmos: com implementações em Pascal e C. 3. ed. São Paulo: Cengage Learning, 2011.

KERNIGHAN, B. W.; RITCHIE, D. M. C: a linguagem de programação. 2. ed. Rio de Janeiro: Campus, 1989.

SILVA, T. R. M. B. Trabalho Prático 1. Notas de aula de Algoritmos e Estruturas de Dados I (CCF 211). Florestal: Universidade Federal de Viçosa, 2026.

ISO/IEC 9899:1999 — Programming languages — C. Genebra: ISO, 1999.

